



**TECNOLÓGICO
NACIONAL DE MÉXICO**

INSTITUTO TECNOLÓGICO SUPERIOR DE JEREZ

Carrera: Ingeniería en Sistemas Computacionales

Nombre de la Materia: Tópicos Selectos Web

**Nombre de la Actividad: Reporte de Practica
(SERVICE WORKER)**

Nombre del Alumno(a): Nayeli Morales García

Número de Control: S19070105

Semestre: 14

Nombre del Docente: Salvador Acevedo Sandoval

Fecha de Entrega: martes 12 de mayo del 2026

INTRODUCCIÓN

En el desarrollo web moderno, la necesidad de crear aplicaciones más rápidas, eficientes y capaces de funcionar sin conexión ha impulsado la implementación de nuevas tecnologías. Entre estas, los Service Workers destacan como una herramienta fundamental para mejorar la experiencia del usuario, permitiendo la gestión de caché, la interceptación de solicitudes de red y la habilitación de funcionalidades offline.

Un Service Worker es un script que se ejecuta en segundo plano dentro del navegador, independiente de la página web. Esto permite realizar tareas como el almacenamiento de recursos en caché, sincronización en segundo plano y el envío de notificaciones push. Gracias a estas características, es posible desarrollar aplicaciones web progresivas (PWA), las cuales ofrecen una experiencia similar a las aplicaciones nativas.

En la presente práctica se aborda la implementación básica de un Service Worker, con el objetivo de comprender su funcionamiento, ciclo de vida y su impacto en el rendimiento y disponibilidad de una aplicación web.

OBJETIVOS

Objetivo general

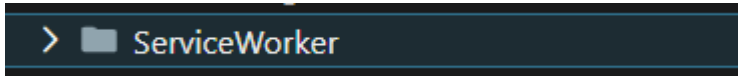
Implementar un Service Worker en una aplicación web básica para comprender su funcionamiento y uso en el almacenamiento en caché.

Objetivos específicos

- Configurar un entorno de desarrollo con Node.js y Express.
- Crear un servidor básico para ejecutar la aplicación.
- Implementar un archivo Service Worker.
- Comprender el ciclo de vida del Service Worker (install, activate, fetch).
- Verificar el funcionamiento en modo offline.

DESARROLLO

Primero ingresamos a nuestro editor de texto y creamos la carpeta del taller con su nombre. Como se ve en la imagen siguiente.



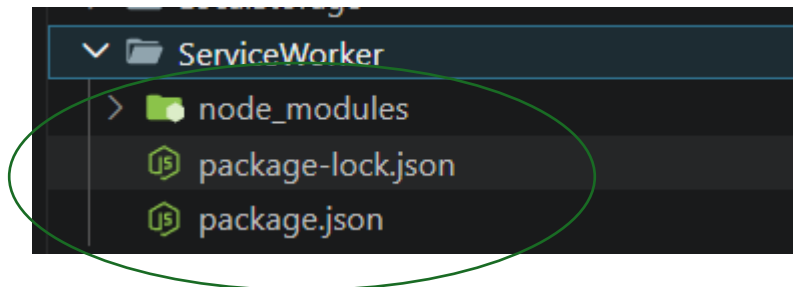
Ya que creamos la carpeta, abrimos una terminal y entramos a esa carpeta. Ya que estemos ahí dentro inicializamos el proyecto para que nos agregue lo que son los nodemodule y los json.

```
PS C:\Users\Wayeli\Desktop\Topicos Programacion WEB\Talleres> cd ServiceWorker
PS C:\Users\Wayeli\Desktop\Topicos Programacion WEB\Talleres\ServiceWorker> npm init -y
Wrote to C:\Users\Wayeli\Desktop\Topicos Programacion WEB\Talleres\ServiceWorker\package.json:

{
  "name": "serviceworker",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

PS C:\Users\Wayeli\Desktop\Topicos Programacion WEB\Talleres\ServiceWorker> |

Después de eso pues en nuestra carpeta se agregaron estos archivos.



Ahora vamos a instalar express, que es el framework que vamos a utilizar.

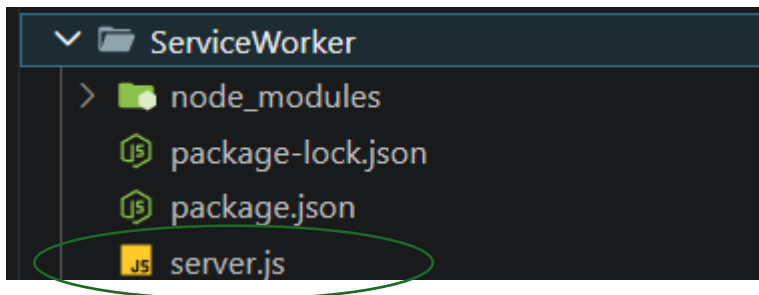
```
PS C:\Users\Wayeli\Desktop\Topicos Programacion WEB\Talleres\ServiceWorker> npm install express

added 65 packages, and audited 66 packages in 3s

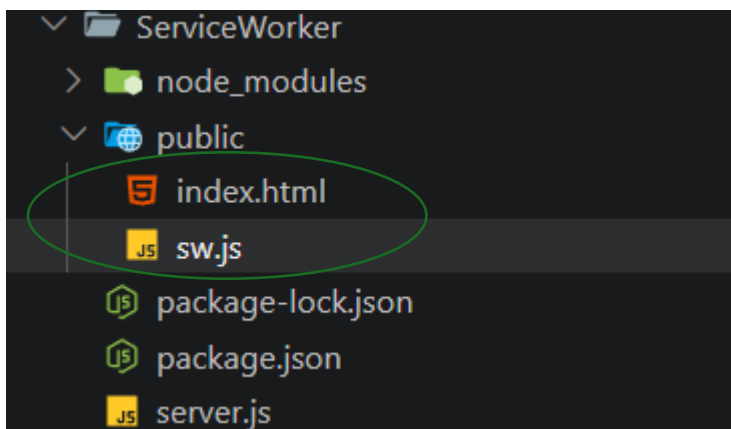
22 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\Wayeli\Desktop\Topicos Programacion WEB\Talleres\ServiceWorker> 
```

Ya que se instaló lo del paso anterior, en la carpeta raíz del proyecto vamos a crear un archivo llamado server.js.



Luego en la misma carpeta raíz, creamos una carpeta llamada public. Dentro de ella dos archivos uno llamado index.html y el otro sw.js.



En el archivo server.js se configuró Express para servir los archivos estáticos y se definió un puerto de ejecución. Esto permite acceder a la aplicación desde el navegador.

```
server.js X
ServiceWorker > .js server.js > ...
1  const express = require("express");
2  const app = express();
3  const PORT = 3000;
4
5
6  app.use(express.static("public"));
7
8  app.listen(PORT, () => {
9    |   console.log(`Servidor corriendo en http://localhost:${PORT}`);
10  | });|
```

En el archivo index.html se agregó la estructura básica de HTML5. Además, se incluyó un script para registrar el Service Worker en el navegador. Este registro permite que el navegador ejecute el archivo sw.js.

```
index.html X
ServiceWorker > public > index.html > html > body > script
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4    <meta charset="UTF-8">
5    <meta name="viewport" content="width=device-width, initial-scale=1.0">
6    <title>Service Worker</title>
7  </head>
8  <body>
9    <h1> Hola Mundo </h1>
10
11    <script>
12      if("serviceWorker" in navigator){
13        navigator.serviceWorker
14          .register("/sw.js")
15          .then(function(registro){
16            console.log("Service Worker registrado", registro.scope);
17          })
18          .catch(function(error){
19            console.log("Error al registrar el service worker", error);
20          });
21      }
22    </script>
23  </body>
24  </html>
```

Ahora abrimos el archivo sw.js y agregamos dos líneas de código, este es para que se vaya guardando los datos del index en el cache.

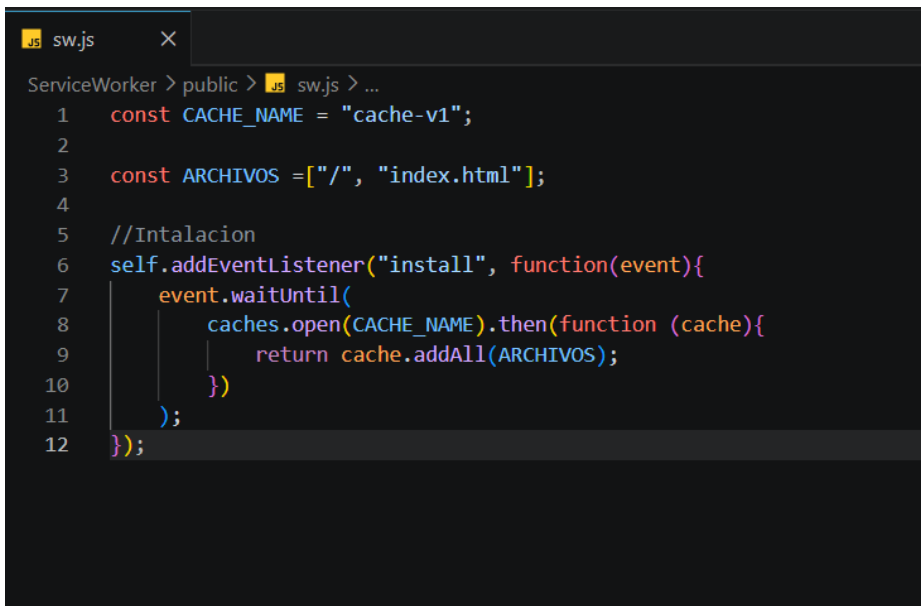


```
ServiceWorker > public > sw.js > ...
1  const CACHE_NAME = "cache-v1";
2
3  const ARCHIVOS =["/", "index.html"];
```

Después ahí mismo en ese archivo, agregamos más código.

Evento install

Este evento se ejecuta cuando el Service Worker se instala por primera vez. Su función es almacenar en caché los archivos necesarios para que la aplicación funcione sin conexión.



```
ServiceWorker > public > sw.js > ...
1  const CACHE_NAME = "cache-v1";
2
3  const ARCHIVOS =["/", "index.html"];
4
5  //Intalacion
6  self.addEventListener("install", function(event){
7      event.waitUntil(
8          caches.open(CACHE_NAME).then(function (cache){
9              return cache.addAll(ARCHIVOS);
10             })
11      );
12  });
```

Evento activate

Se encarga de activar el Service Worker y limpiar versiones antiguas del caché, asegurando que solo se utilicen los recursos más recientes.

```
//Activacion
self.addEventListener("activate", function(event){
    event.waitUntil(
        caches.keys().then(function(nombres){
            return Promise.all(
                nombres.filter(function(nombre){
                    return nombre !== CACHE_NAME;
                }).map(function(nombre){
                    return caches.delete(nombre);
                })
            );
        });
    );
});
```

Evento fetch

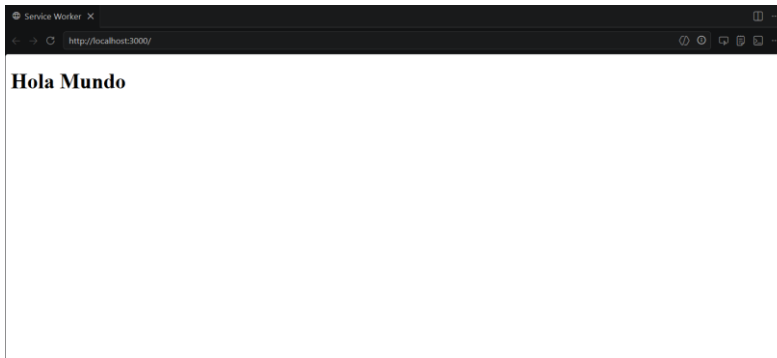
Intercepta las solicitudes de red. En este caso, se configuró para responder con los recursos almacenados en caché, permitiendo que la aplicación funcione incluso sin conexión a internet.

```
//Fetch-Cache First
self.addEventListener("fetch", function(event){
    event.respondWith(
        caches.match(event.request).then(function(respuesta){
            if(respuesta){
                return respuesta;
            }
            return fetch(event.request);
        })
    );
});
```

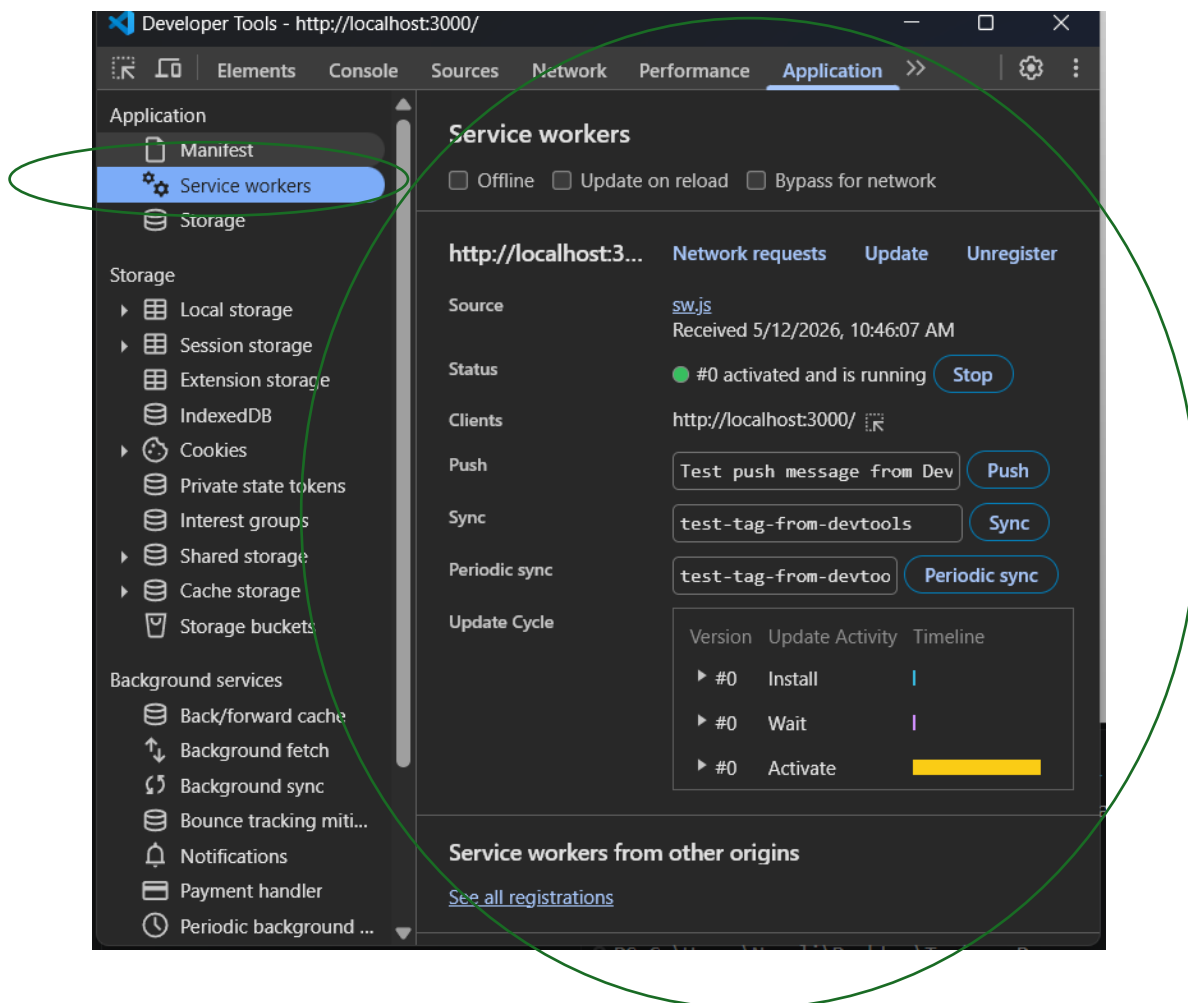
Se ejecutó el servidor mediante un comando en la terminal. Al acceder a la dirección proporcionada (localhost), se mostró la página web correctamente, indicando que la configuración fue exitosa.

```
PS C:\Users\Nayeli\Desktop\Topicos Programacion WEB\Talleres\ServiceWorker> node server.js
Servidor corriendo en http://localhost:3000
```

Ahora nos abre el navegador y nos muestra el index que configuramos que es Hola Mundo. Hasta aquí todo va bien.



Ahora le damos click derecho en algún lugar de la pantalla y en inspeccionar. Nos va a mostrar lo de la imagen siguiente y si nos ubica en otra pestaña pues buscamos donde diga application y ingresamos ahí y en el menú de la barra lateral izquierda nos tiene que aparecer Service Worker y nos aparecen todas sus características del servicio y que esta activo el servicio y funcionando.



Ahora checamos que este guardando lo del cache en el mismo menú de barra lateral izquierda bajamos y buscamos Cache Storage y presionamos para que se deslice una pestañita que diga cache-v1 y tiene que aparecer los datos del index fechas y toda la información.

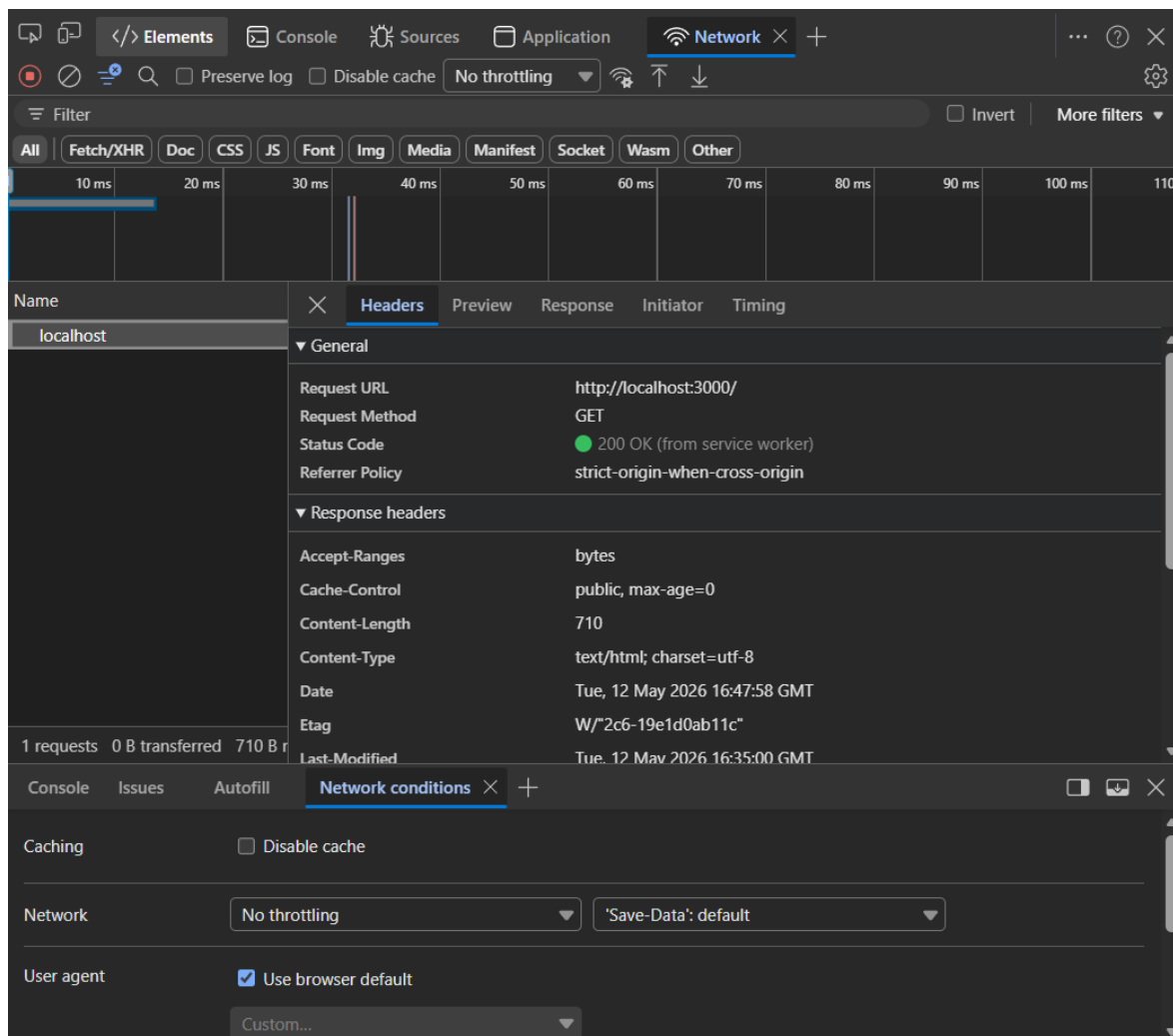
The screenshot shows the Chrome DevTools Application tab. In the left sidebar, under 'Storage', 'Cache storage' is expanded, and 'cache-v1 - http://loc...' is selected. The main panel displays the cache for 'http://localhost:3000' with a table of entries. A green oval highlights the table. Below the table, it says 'No cache entry selected'. The console at the bottom shows a 404 error for a favicon and a log for a Service Worker.

#	Name	Respo...	Conte...	Conte...	Time ...	Vary ...
0	/	basic	text/h...	710	12/5/...	
1	/index.html	basic	text/h...	710	12/5/...	

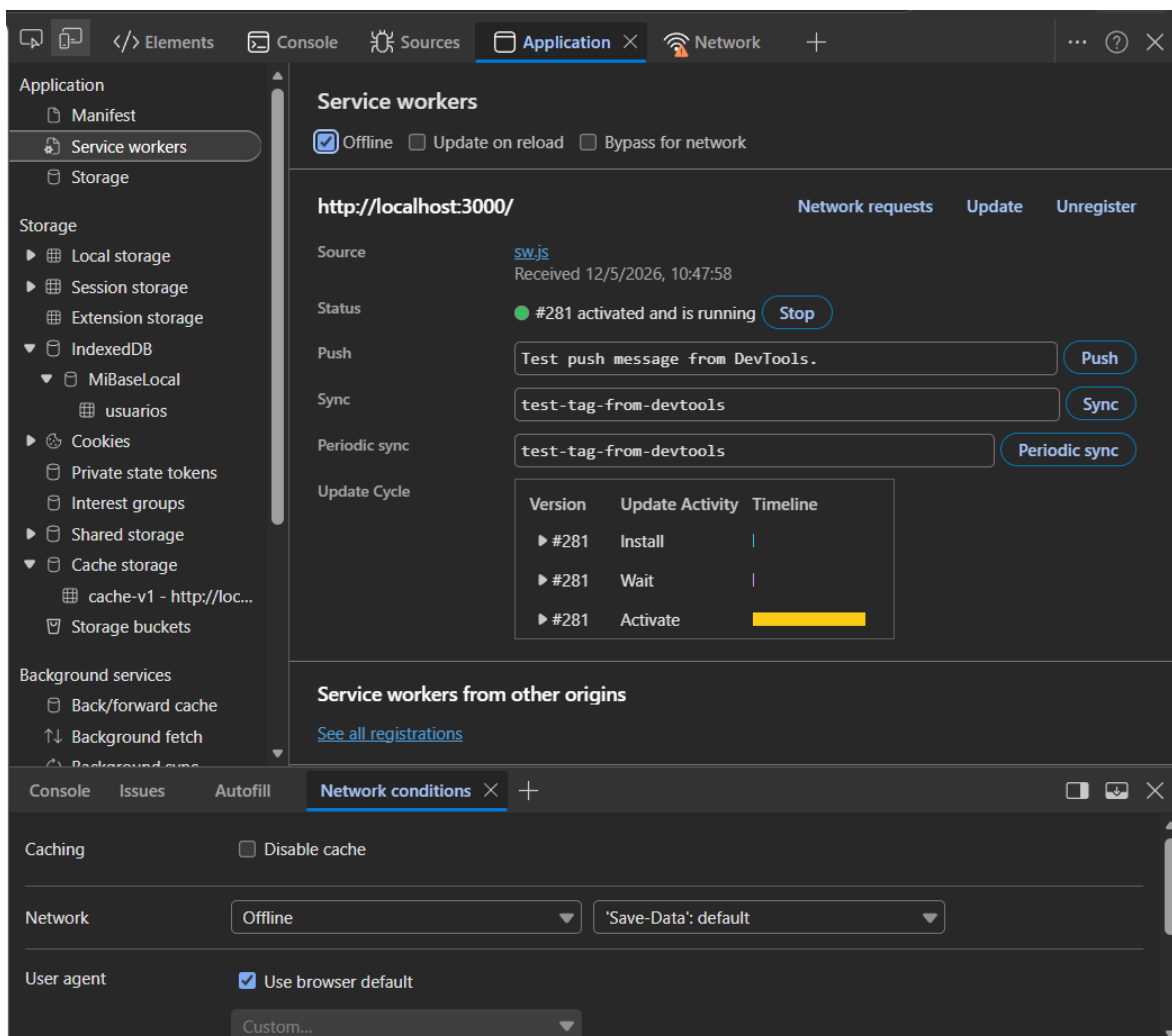
Failed to load resource: the server responded with a status of 404 (Not Found) :3000/favicon.ico:1

Service Worker registrado <http://localhost:3000/> (.index):16

Ahora buscamos la pestaña Network en el menú de pestañas de arriba y presionamos en la que indicamos. Si sale en blanco se le da en read un botón azul y te aparece el localhost y le doy click y te aparecerá todo lo que esta funcionando y la url donde está ubicada y demás características.



Se activó el modo offline desde las herramientas del navegador y se recargó la página. La aplicación continuó funcionando correctamente, demostrando que el Service Worker estaba operando de manera adecuada.



CONCLUSIÓN

En esta práctica se logró implementar correctamente un Service Worker en una aplicación web básica, comprendiendo su funcionamiento y su importancia en el desarrollo moderno. Se aprendió que los Service Workers permiten mejorar el rendimiento de las aplicaciones mediante el uso de caché y brindan la capacidad de funcionar sin conexión a internet.

Asimismo, se comprendió el ciclo de vida del Service Worker, destacando los eventos de instalación, activación y manejo de solicitudes. Esta tecnología resulta fundamental en el desarrollo de aplicaciones web progresivas, ya que mejora significativamente la experiencia del usuario.

Finalmente, esta práctica permitió reforzar conocimientos sobre servidores, manejo de archivos y funcionamiento interno del navegador, lo cual es esencial para el desarrollo web actual.